

SafeSend

A poison-aware payment router for EVM.
Makes the address-poisoning paste cancellable.

3rd Web Hack – unaudited testnet prototype – Solidity · Foundry · React

no owner · no fees · no upgradeability · on-chain enforcement

01

Anyone can make your history lie

- Wallets render addresses as `0x7099...79c8` — only the first and last 4 hex are ever checked.
- Attackers vanity-generate a **lookalike** matching those same edges of a real payee.
- They inject it into your history: a **zero-value transferFrom** (succeeds on USDT-style tokens with **no allowance**) plus a dust transfer.
- You copy your "last counterparty", hit send — **irreversible**. Documented single-victim losses reach tens of millions; campaigns run continuously because they cost the attacker ~nothing.

02

Same fingerprint the wallet shows — enforced on-chain

SafeSend computes $fp = first4 \ll 16 \mid last4$ of every recipient and routes accordingly:

Recipient	Routing
Verified payee	Instant transfer — no escrow
Unknown payee	Sender-cancellable escrow · recipient claims after cooldown
Fingerprint collides with a verified payee	24h quarantine + on-chain LookalikeFlagged event · always cancellable

03

The contract — a router, not a vault

send(to, token, amount)

Verified → instant. Lookalike → 24h lock + flag.
Unknown → escrow for sender's cooldown (60s–7d).

cancel(id) sender only, any time → full refund
claim(id) recipient only, after unlock → pays out
and verifies them

reclaim(id) after unlock + 30d — dead-letter
recovery

addPayee / removePayee / setCooldown — the
sender's own book. No owner. No admin. No fees.

OpenZeppelin SafeERC20 + ReentrancyGuard · checks-effects-interactions · no external dependencies

04

Watch the attack, then watch it fail

- **Attacker console** (Anvil only): pick a payee → generated lookalike → **poison** (zero-value transferFrom) + **dust** (impersonation).
- **History** flags both injected rows — the lookalike now "looks like" the real payee.
- Paste it into **Send** → **LOOKALIKE badge** → contract quarantines + emits the flag.
- **Pending** → cancel → full refund. A raw transfer here is gone forever.
- Same flow to the real payee → **instant**. New contact → escrow → they claim → **verified forever**.

05

Engineering rigor

- **45 Foundry tests:** unit, negative (NotRecipient / NotSender / Locked / TooEarly ...), event assertions.
- **Fuzz** (512 runs): random amounts, recipients, timings across every transition.
- **Invariants** (128 × depth 32): router solvency for ERC-20 + ETH; nextId monotonic; verified recipients can never sit in escrow.
- **100% line coverage** on SafeSend.sol · `forge fmt` clean · gas snapshot.
- UI mirrors the contract's fingerprint exactly — parity-tested against Solidity output.

Honest limitations

- **Detection, not prevention** — only protects sends routed through SafeSend; raw `transfer()` is untouched.
- **4-hex fingerprint** — covers the common display convention; an attacker matching ≥ 6 chars can still slip by.
- **addPayee trusts the sender** — out-of-band verification is the human's job; claim-to-verify is the safer path.
- **Unaudited prototype.** Prior art: REVERSO / ERC-20R (arbitrated reversals — we use a deterministic rule, no judges), Argent trusted contacts (ours self-grows from deliveries), explorer tags (off-chain — ours enforces on-chain).

The most common way people lose to poisoning is now a refund

Copy-from-history is a UI habit. SafeSend turns its worst case — a lookalike paste — into a 24-hour escrow the attacker can never claim and the sender can always cancel.

- Runs today: `make anvil && make deploy-local && make seed && make web`
- Base Sepolia deploy path included; attack tooling confined to local Anvil, labeled for education.
- Next: EIP-712 permit UX, wallet-level fingerprint warnings, broader collision sets.